

Problem1.

1-a.NeRF：全名為 **Neural Radiance Field**，目的將一系列 2D 的圖片中學習到 3D 場景，再渲染出某個角度的 2D 圖片，NeRF 的輸入為一個 5 維的函數，包含該圖片所拍攝的視角位置以及視角方向，輸出是該視角的 2D 渲染圖片，包含某個座標的顏色以及不透明度。

1-b.我覺得在輸入時同時輸入該圖片的視角是很重要的一環，從論文中可以看到若沒有輸入視角會導致輸出圖片中的鏡面反射部分效果很差，另外對拍攝視角位置做 **position encoding** 也是模型中的關鍵部分，作者也發現了若直接將位置和角度輸入的話會造成輸出的圖片相對模糊，而 **position encoding** 能改善這點。

1-c.

優點：

- (1).可以應用於絕大部分的場景，因為該方式對資料本身沒有任何限制。
- (2).網路由 MLP 構成，相對小且占用較少記憶體。

缺點：

- (1).一個網路只能描述一個場景(即一組圖片)，且需要大量的圖片來訓練，訓練時間較長。
- (2).模型訓練初期容易不穩定。

2.DVGO 將整個 3D 場景表示為兩個 **voxel grid**，分別為顏色+視角和密度，對於場景中的每一個點用插值法得到該點的 **voxel grid**，而在訓練時分為兩個階段，分別為 **coarse** 和 **fine**，**coarse** 會訓練密度以及顏色的 **voxel grid**，藉由少量的學習確定整個場景大概的樣貌，而 **fine** 階段則可藉由 **coarse** 中學習的密度 **voxel grid** 來去除無關緊要的点，並在測試時也能利用此密度 **voxel grid** 來減少不須重現的点(如空白處和被遮住的部分)。DVGO 訓練時間短的原因主要有用 **coarse** 過程來減少需訓練的範圍和用插值法來計算整個場景中未知訊息的部分(原始 NeRF 使用 MLP)。

3.

Setting	PSNR	SSIM	LPIPS
Original setting in DVGO	35.1738	0.9745	0.0413
Change the coarse training steps to 1000(5000 in default)	35.2279	0.9744	0.0412

可以看到改變 **coarse** 過程中的訓練步數對結果沒有什麼影響，可能要再將其數值調得更小才會有所改變，這也側面反映出了 **coarse** 過程的效用，即少量訓練即可達到很好的 **density voxel grid**。

PSNR 用於計算一張影響經過某種方式重建後的品質，**PSNR** 越大，則代表欲比較之圖片和原圖差距越小，其細節越逼真。

SSIM 則計算兩張圖片的結構相似度，**SSIM** 更側重人眼的感覺，如亮度等指標，而對細部圖片的品質則較不在意。

LPIPS 則模擬人類辨認兩張圖片是否相同時的運作原理，藉由用神經網路提取圖片特徵的方式來判斷兩張圖片的相似程度，其值越小代表兩張圖片越相近。

Problem2

1.

I choose BYOL for SSL pretraining. The data augmentation for SSL is shown in the following figure. In the pretraining process, I choose Adam as optimizer with the learning rate $3e-4$ and no learning rate scheduler. The total training epochs is 1000 and the batchsize is 64.

```
def expand_greyscale(t):  
    return t.expand(3, -1, -1)  
self.transform = transforms.Compose([  
    transforms.Resize(128),  
    transforms.CenterCrop(128),  
    transforms.ToTensor(),  
    transforms.Lambda(expand_greyscale)  
)
```

2.

For the downstream task, I set the training epochs to 100 and the batchsize to 32. The optimizer is Adam with the learning rate $2e-4$ and the weight decay $1e-5$ and no learning rate scheduler.

Setting	Pre-training (Mini-ImageNet)	Fine-tuning (Office-Home dataset)	Validation accuracy (Office-Home dataset)
A	-	Train full model (backbone + classifier)	0.19951
B	w/ label (TAs have provided this backbone)	Train full model (backbone + classifier)	0.27586

C	w/o label (Your SSL pre-trained backbone)	Train full model (backbone + classifier)	0.5296
D	w/ label (TAs have provided this backbone)	Fix the backbone. Train classifier only	0.29557
E	w/o label (Your SSL pre-trained backbone)	Fix the backbone. Train classifier only	0.48769

From the above table shows, setting C performs the best and the accuracy is . I think that the reason is that the backbone TA provided trained in less epochs than my SSL pre-trained backbone. And the setting A has lowest performance, I think the reason is that I only trained 100 epochs in fine-tuning. Last, train full model can get higher performance than train classifier only, I think that the result is predictable since that the backbone is trained on different dataset and finetune it can let the backbone fit the target dataset.